Модуль 2. Дополнительные возможности Pandas

2.1. Работа со временем и графиками
- Работа со временем
- Понятие эпоха (epoch)
- Секунды, struct_time и преобразование друг в друга
- Строки временных меток
- Использование работы со временем для построения визуализаций
- Использование функционала Pandas для создания графиков

2.2. Написание собственных функций для работы с Pandas
- Метод apply()
- Создание визуализаций с помощью собственных функций
- Работа с выбросами после применения собственной функции
- Метод .pipe(): использование собственного функционала Pandas
- Отличия методов .apply() и .pipe

Цели модуля

- Познакомимся с основами работы со временем в Python.
- Поймем принципы визуализации временных рядов.
- Узнаем, как использовать Pandas для визуализации данных.
- Научимся писать собственные функции для Pandas.

2.1. Работа со временем и графиками

Часто аналитики данных работают с данными, которые имеют временные метки, например, о динамике ВВП разных стран за последние 50 лет. В таком случае для построения визуализаций и моделей необходимо хорошо владеть методами работы со временем.

Но зачастую при работе с датасетами в Pandas возникает необходимость повторять одни и те же действия многократно. Иногда для их выполнения не хватает стандартного набора методов, встроенных в библиотеку.

Pandas же позволяет использовать собственные кастомные функции и применять их к датасету. Посмотрим, как это работает.

```
# Импортируем все библиотеки, которые будем использовать

from datetime import datetime, timedelta

import matplotlib.pyplot as plt

import numpy as np

import pandas as pd

import math

import missingno as msno

import seaborn as sns

import os


%matplotlib inline

palette = plt.get_cmap('Set2')
```

Мы используем те же датасеты, что и в предыдущем модуле. Импортируем их перед работой:

```
# Импортируем данные

kiva_loans =
pd.read_csv('https://raw.githubusercontent.com/ElijahSum/mipt_
visualization/master/week_02_visualization/kiva_loans_week2',
index_col=0)

mpi =
pd.read_csv('https://raw.githubusercontent.com/ElijahSum/mipt_
visualization/master/week_01_visualization/data/kiva_mpi_regio
n_locations.csv')
```

Работа со временем

Чтобы эффективно работать с представленными данными, нам необходимо понимать принципы работы со временем. Например, мы хотим создать график динамики изменения количества займов по месяцам. С помощью работы со временем мы можем агрегировать все наши данные по месяцам и посчитать их количество.

Встроенные библиотеки для работы со временем и интервалами времени в Python:

- time,
- datatime,
- calendar,
- timeit.

Для решения наших задач чаще всего используют time.

Методы, связанные со временем, распределены во встроенных модулях Python 3 довольно компактно. Однако наблюдаются и некоторые пересечения. Рассмотрим пример:

```
import time
```

Понятие эпоха (epoch)

Работа с модулем time в существенной мере зависит от операционной системы. Время в библиотеке привязано к фиксированной начальной точке — **эпохе** (epoch). Узнаем эту начальную точку:

```
time.time(0)
```

time.struct_time(tm_year=1970, tm_mon=1, tm_mday=1, tm_hour=0, tm_min=0, tm_sec=0, tm_wday=3, tm_yday=1, tm_isdst=0)

Начальная точка отсчета (epoch) в Windows и Unix-системах — 1 января 1970 г. Функция gmtime() вернула объект именованного кортежа struct_time.

С помощью функции time() время, прошедшее с этой начальной точки, можно также вывести в секундах (seconds since the epoch):

```
time.time()
```

```
1701539900.3941581
```

Так как точка epoch для разных операционных систем может отличаться, число секунд, возвращаемое функцией time() также может быть различным.

Время до точки epoch тоже существует, но значения секунд seconds since the epoch отрицательны. Если мы передадим функции gmtime отрицательное значение секунд, мы перенесемся в прошлое относительно момента времени epoch:

```
time.gmtime(-10**8)
```

```
time.struct_time(tm_year=1966, tm_mon=10, tm_mday=31,
tm_hour=14, tm_min=13, tm_sec=20, tm_wday=0, tm_yday=304,
tm_isdst=0)
```

Секунды, struct_time и преобразование друг в друга

Итак, модуль time оперирует двумя основными типами объектов: struct_time и секундами с начала эпохи. Для взаимных преобразований используют следующие функции:

- gmtime(): из секунд в struct_time для UTC;
- localtime(): из секунд в struct_time для местного времени;
- calendar.timegm() (не модуль time): из struct_time для UTC в секунды;
- mktime(): из struct_time местного времени в секунды.

UTC — стандартное обозначение всемирного координированного времени без учета часовых поясов. Начало отсчета эпохи привязано к UTC, то есть не зависит от местного времени. Поэтому UTC удобнее использовать для общения с внешним миром.

```
time.gmtime() # UTC-time
```

```
time.struct_time(tm_year=2023, tm_mon=12, tm_mday=2,
tm_hour=17, tm_min=58, tm_sec=20, tm_wday=5, tm_yday=336,
tm_isdst=0)
```

```
time.localtime() # Локальное время на сервере (при
использовании на своей машине) вернет ваше локальное время
```

```
time.struct_time(tm_year=2023, tm_mon=12, tm_mday=2,
tm_hour=17, tm_min=58, tm_sec=20, tm_wday=5, tm_yday=336,
tm_isdst=0)
```

Если отсутствует аргумент функций gmtime() и localtime(), то возвращаем значение для текущего времени — UTC и местное время.

Для преобразования объекта struct_time в секунды можно или передать сам объект, или кортеж целых чисел. Порядок элементов в кортеже:

1. Год, tm_year.
2. Месяц, tm_mon: целое число (1 — январь, 12 — декабрь).
3. День месяца, tm_day.
4. Час, tm_hour: целое число в диапазоне от 0 до 23.
5. Минута, tm_min.
6. Секунда, tm_sec.
7. День недели, tm_wday: целое число от 0 (понедельник) до 6 (воскресенье).
8. День года, tm_yday.
9. Целочисленный флаг для учета перехода на летнее время (daylight saving time, DST), tm_isdst: 1 — переход на летнее время учитывается, 0 — не учитывается, -1 — неизвестно.

```
time.mktime((2015, 10, 21, 7, 28, 0, 2, 294, -1))
```

```
1445412480.0
```

```
time.mktime(time.localtime())
```

```
1701539900.0
```

Заметим, что в отличие от представления в виде секунд, struct_time не хранит составляющие времени, которые меньше секунды.

Удобство struct_time в том, что это именованный кортеж. Значит, можно писать более ясный код — элементы объекта можно вызвать по ключам с говорящими названиями.

```
t = time.localtime()

'Итак, на дворе {}-й год.'.format(t.tm_year)
```

```
'Итак, на дворе 2023-й год.'
```

Строки временных меток

Выше мы использовали функции, которые возвращают время в секундах, часах, годах. Однако чаще мы работаем с датами. Для этого в библиотеках есть специальные функции для преобразования в дату.

Распространенная задача — преобразование объектов указанных типов в строки вида timestamp, например, Mon Dec 2 18:30:20 2019. Для этого применяются функции:

- ctime() — принимает время в секундах.
- asctime() — принимает struct_time (по умолчанию используется местное время).

```
time.ctime(time.time()) # Преобразует время в секундах в timestamp для местного времени
```

```
'Sat Dec  2 17:58:20 2023'
```

```
time.asctime() # Аналогично time.asctime(time.localtime())
```

```
'Sat Dec  2 17:58:20 2023'
```

```
time.asctime(time.gmtime()) # Время UTC
```

```
'Sat Dec  2 17:58:20 2023'
```

Хотя строковый вывод функций ctime() и asctime() довольно удобен, может потребоваться альтернативный формат. Для гибкого форматирования в библиотеку time включена функция strftime(). Функция принимает строку шаблона форматирования со спецификаторами, а также сам объект времени.

```
time.strftime('%d.%m.%Y', time.localtime())

'02.12.2023'
```

```
import locale

locale.setlocale(locale.LC_TIME)

time.strftime('Текущее время: %c', time.localtime())

'Текущее время: Sat Dec  2 17:58:20 2023'
```

Использование работы со временем для построения визуализаций

Представим, что мы хотим узнать динамику изменения количества кредитных денег, выданных в месяц. Для этого необходимо сгруппировать имеющиеся данные по месяцам, а затем сложить все средства, выданные в определенный месяц.

Для решения подобных задач в Pandas предустановлен TimeStamp — тип данных, позволяющий работать с датами по рассмотренному выше принципу.

```
# Снова посмотрим на наш датасет

kiva_loans.head()
```

funded_amount	loan_amount	activity	sector	country	region	partner_id	funded_time	lender_count	borrower_genders	
0	300.0	300.0	Fruits & Vegetables	Food	Pakistan	Lahore	247.0	2014-01-02 10:06:32+00:00	12	female
1	575.0	575.0	Rickshaw	Transportation	Pakistan	Lahore	247.0	2014-01-02 09:17:23+00:00	14	female, female
2	150.0	150.0	Transportation	Transportation	India	Maynaguri	334.0	2014-01-01 16:01:36+00:00	6	female
3	200.0	200.0	Embroidery	Arts	Pakistan	Lahore	247.0	2014-01-01 13:00:00+00:00	8	female
4	400.0	400.0	Milk Sales	Food	Pakistan	Abdul Hakeem	245.0	2014-01-01 19:18:51+00:00	16	female

```
# Проверяем тип данных в столбце с датами

type(kiva_loans.funded_time[0])
```

```
str
```

```
kiva_loans.funded_time[0]
```

```
2014-01-02 10:06:32+00:00
```

Преобразуем имеющийся столбец с датами в формат Timestamp, предустановленный в библиотеке Pandas.

```
kiva_loans.funded_time =
pd.to_datetime(kiva_loans.funded_time,

               format='%Y-%m-%d %H:%M:%S.%f')
```

Посмотрим на результаты преобразования:

```
type(kiva_loans.funded_time[0])
```

```
pandas._libs.tslibs.timestamps.Timestamp
```

Как мы видим, теперь наш тип данных преобразован из str в Timestamp. А значит, нам доступны все методы работы с датами. Например, с помощью метода .month мы можем посмотреть, в каком месяце произошло событие:

```
kiva_loans.funded_time[0].month
```

```
1
```

Используем полученные даты для решения практических задач. Напоминаем, что в нашем датасете две переменные, которые нам интересны для решения задачи:

- funded_amount — сумма, выплаченная Kiva агенту на местах (USD);
- loan_amount — сумма, выплаченная полевым агентом.

```
# Создадим переменную, в которой будет указан только год и месяц взятия займа. Ее мы будем использовать для группировки

kiva_loans['YearMonth'] = pd.to_datetime(kiva_loans['funded_time']).apply(lambda x: '{year}-{month}'.format(year=x.year, month=x.month))


# Сгруппируем данные по созданной выше переменной, суммируем все численные значения

# Использование метода .iloc[:-1, :] необходимо, поскольку последним элементом остается NaN — данные без указанной даты. Это портит анализ

loans_per_month = kiva_loans.groupby(by=['YearMonth'], as_index=False).sum(numeric_only=True).iloc[:-1, :]


# Преобразовываем тип данных переменной YearMonth в дату
```

```
loans_per_month.YearMonth =
pd.to_datetime(loans_per_month.YearMonth,
format='%Y-%m').dt.to_period('M')


# Сортируем, оставляем только интересующие нас столбцы

loans_per_month = loans_per_month.sort_values(by='YearMonth')

loans_per_month = loans_per_month.drop(['partner_id',
'lender_count'], axis=1)

loans_per_month.head(5)
```

YearMonth	funded_amount	loan_amount	
0	2014-01	5933175.0	5933175.0
4	2014-02	9599075.0	9599075.0
5	2014-03	13200400.0	13200400.0
6	2014-04	10331475.0	10331475.0
7	2014-05	10595075.0	10595075.0

Использование функционала Pandas для создания графиков

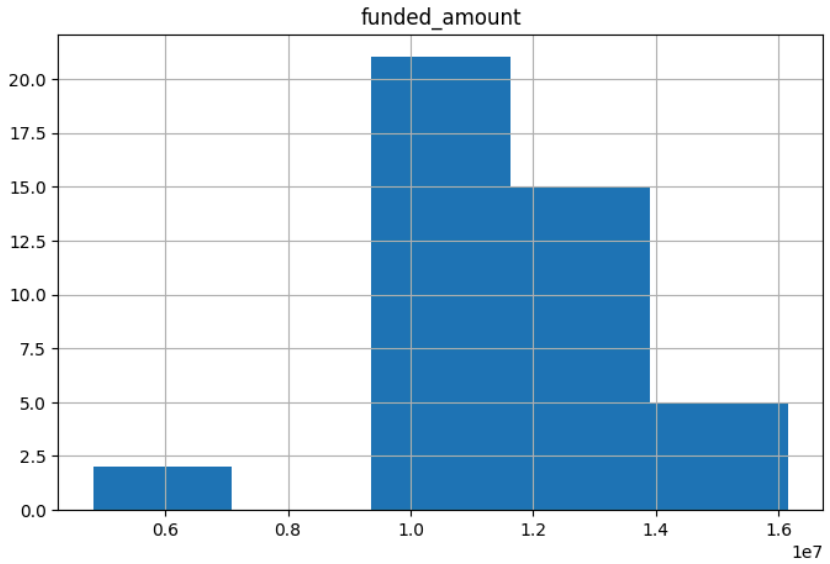
В библиотеке Pandas есть свой модуль pandas.plot, позволяющий быстро рисовать простые графики. Он рассчитан на создание быстрых и простых визуализаций по ходу анализа, чтобы получить более полное представление об имеющихся данных.

Разберем его основной функционал.

Визуализация распределения переменной. Чтобы создать простейший график, после названия датафрейма необходимо написать .plot.bar, где вместо .bar может стоять любой интересующий нас график. Иногда начальное .plot можно опустить, например, при создании гистограммы:

```
loans_per_month.hist('funded_amount', bins=5, figsize=(8, 5))
```

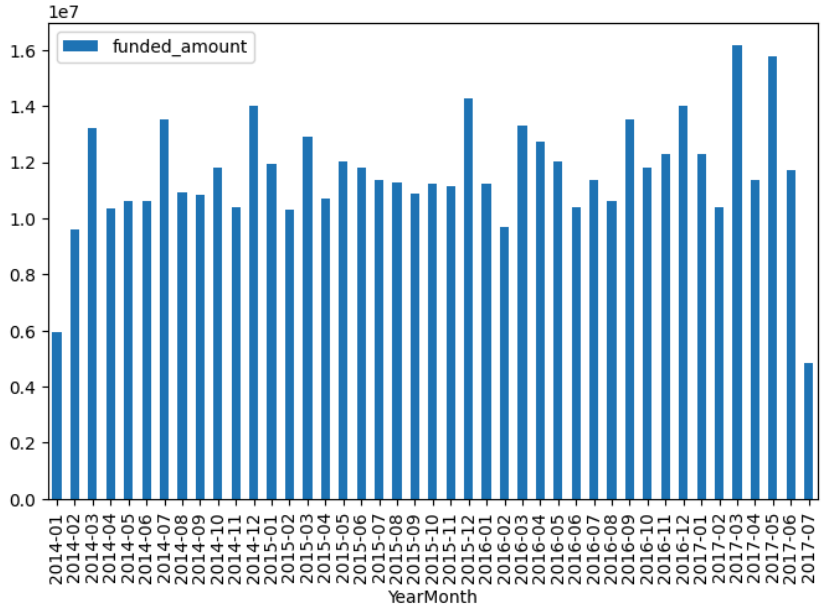
```
array([[<Axes: title={'center': 'funded_amount'}>]],
dtype=object)
```



Создание визуализаций с помощью Pandas. Для использования некоторых графиков в библиотеке Pandas необходимо использовать модуль .plot перед указанием типа графика:

```
loans_per_month.plot.bar(x='YearMonth', y='funded_amount',
figsize=(8, 5))
```

```
<Axes: xlabel='YearMonth'>
```

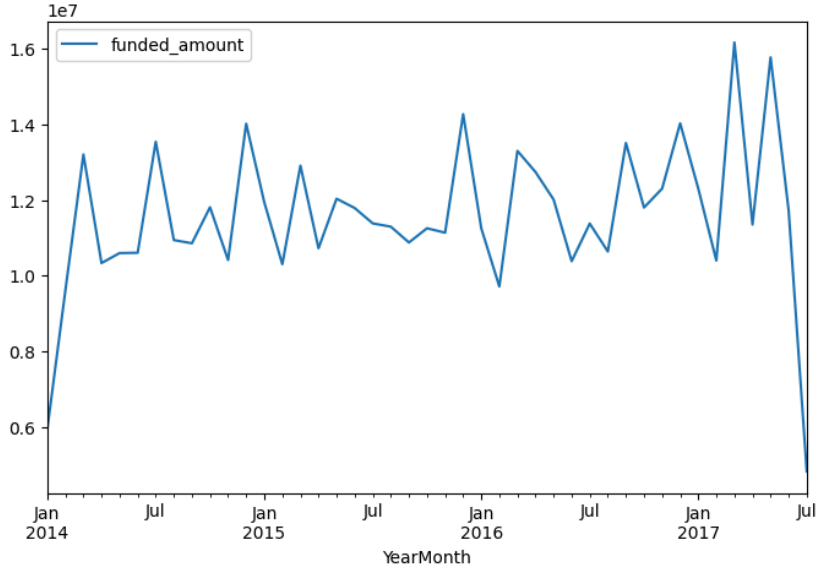


Таким образом, мы построили график изменения объема выданных ссуд по месяцам. Какие зависимости вы видите на графике?

Если не уточнять, какой график будет использован, то по умолчанию всегда строится lineplot:

```
loans_per_month.plot(x='YearMonth', y='funded_amount',
figsize=(8, 5))
```

```
<Axes: xlabel='YearMonth'>
```

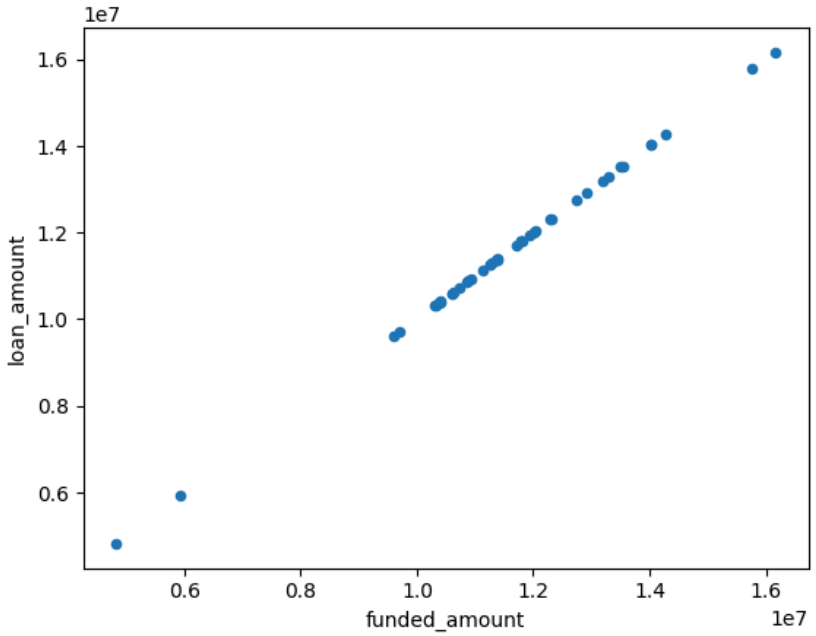


Поскольку у нас есть две численные переменные, которые напрямую связаны друг с другом, построим график созависимости двух переменных, используя .plot.scatter().

Если значения на таком графике расположены близко к диагонали, то существует очень высокая корреляция между двумя значениями. В зависимости от диагонали, корреляция может быть как негативной, так и позитивной. В нашем случае значения расположены очень близко — корреляция близка к единичной.

```
loans_per_month.plot.scatter(y='loan_amount',
x='funded_amount')
```

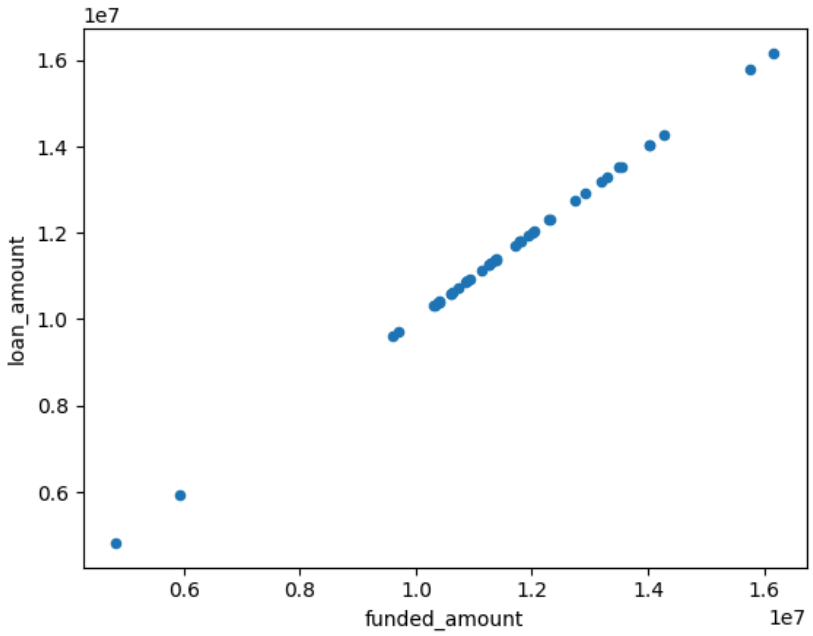
```
<Axes: xlabel='funded_amount', ylabel='loan_amount'>
```



Все те же самые действия можно сделать, воспользовавшись параметром kind в методе .plot.

```
loans_per_month.plot(kind='scatter', y='loan_amount',
x='funded_amount')
```

```
<Axes: xlabel='funded_amount', ylabel='loan_amount'>
```



2.2. Написание собственных функций для работы с Pandas

Все проделанные действия не уникальны для конкретно нашего датасета. В рамках большого проекта многие из этих операций нужно повторять много раз, например, чтобы преобразовать данные из типа строки (str) в дату (Timestamp). Такая ситуация может возникнуть, когда мы работаем с несколькими датасетами в рамках одного проекта.

Чаще при импорте данных возникает проблема с типом импортированных данных — числовые значения интерпретируются как str. Для ее решения удобно написать функцию, а затем многократно ее использовать. Pandas предлагает удобные инструменты работы с такими функциями с помощью метода .apply.

```
kiva_loans_pandas =
pd.read_csv('https://raw.githubusercontent.com/ElijahSum/mipt_
visualization/master/week_02_visualization/kiva_loans_week2',
index_col=0)
```

Метод apply()

Метод .apply() позволяет применять заданную функцию к каждому элементу pandas.Series или pandas.DataFrame.

Автоматизируем то, что делали ранее, и напишем функцию, которая поменяет тип данных столбца на дату:

```
def str_to_date(series):

 return series.apply(lambda x: pd.to_datetime(x,
format='%Y-%m-%d %H:%M:%S.%f'))



print("Тип данных funded_time до применения функции:",
type(kiva_loans_pandas.funded_time[0]))

kiva_loans_pandas.funded_time =
str_to_date(kiva_loans_pandas.funded_time)

print("Тип данных funded_time после применения функции:",
type(kiva_loans_pandas.funded_time[0]))
```

```
Тип данных funded_time до применения функции: <class 'str'>

Тип данных funded_time после применения функции: <class
'pandas._libs.tslibs.timestamps.Timestamp'>
```

```
kiva_loans_pandas.head()
```

funded_amount	loan_amount	activity	sector	country	region	partner_id	funded_time	lender_count	borrower_genders	
0	300.0	300.0	Fruits & Vegetables	Food	Pakistan	Lahore	247.0	2014-01-02 10:06:32+00:00	12	female
1	575.0	575.0	Rickshaw	Transportation	Pakistan	Lahore	247.0	2014-01-02 09:17:23+00:00	14	female, female

2	150.0	150.0	Transportation	Transportation	India	Maynaguri	334.0	2014-01-01 16:01:36+00:00	6	female
3	200.0	200.0	Embroidery	Arts	Pakistan	Lahore	247.0	2014-01-01 13:00:00+00:00	8	female
4	400.0	400.0	Milk Sales	Food	Pakistan	Abdul Hakeem	245.0	2014-01-01 19:18:51+00:00	16	female

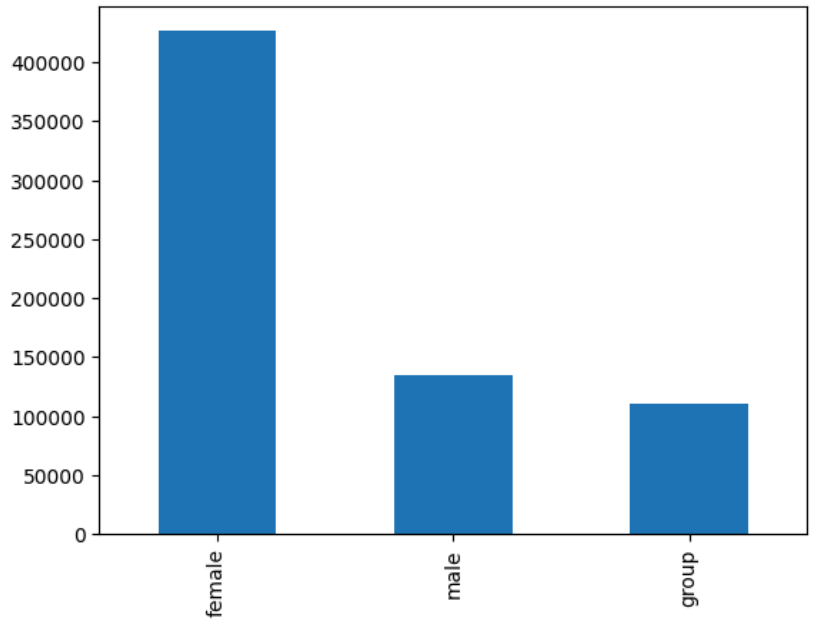
Создание визуализаций с помощью собственных функций

Можно создавать более сложные функции с помощью того же функционала. В 1 модуле мы писали код, который заменял значения на «группа», если ссуда предоставлялась нескольким людям. Теперь сделаем то же самое с помощью нового функционала:

```
def gender_get(series):

 return series.apply(lambda x: 'female' if x=='female' else 'male' if x=='male' else 'group')



gender_distribution = gender_get(kiva_loans_pandas['borrower_genders']).value_counts()
```

```
gender_distribution.plot.bar()
```

```
<Axes: >
```



Работа с выбросами после применения собственной функции

Узнаем, насколько отличается размер funded_amount от loan_amount. Поскольку анализ scatter plot не дал результатов, создадим новую переменную путем вычитания одного столбца из другого:

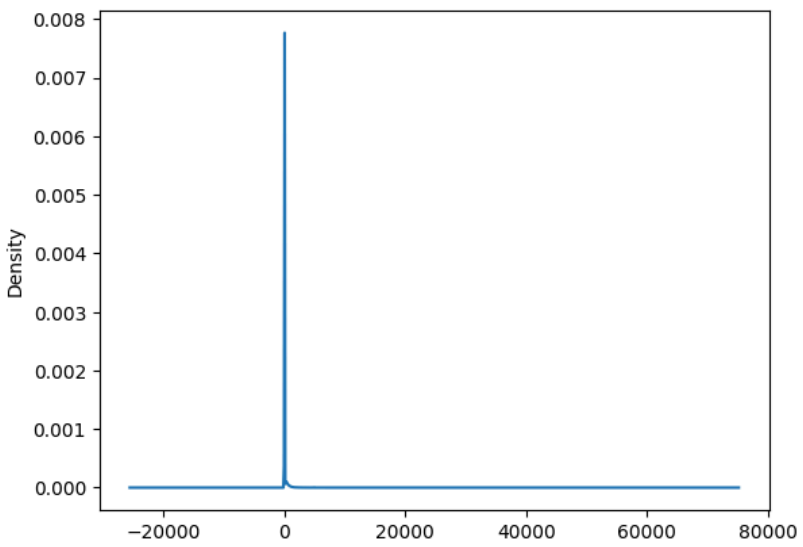
```
def subtracted_amount(df, first_col, second_col, result):

 df[result] = df[first_col] - df[second_col]

 return df


kiva_loans_pandas = subtracted_amount(kiva_loans_pandas,
'loan_amount', 'funded_amount', 'unfunded_amount')

kiva_loans_pandas.head(5)
```

funded_amount	loan_amount	activity	sector	country	region	partner_id	funded_time	lender_count	borrower_genders	unfunded_amount	
0	300.0	300.0	Fruits & Vegetables	Food	Pakistan	Lahore	247.0	2014-01-02 10:06:32+00:00	12	female	0.0
1	575.0	575.0	Rickshaw	Transportation	Pakistan	Lahore	247.0	2014-01-02 09:17:23+00:00	14	female, female	0.0
2	150.0	150.0	Transportation	Transportation	India	Maynaguri	334.0	2014-01-01 16:01:36+00:00	6	female	0.0
3	200.0	200.0	Embroidery	Arts	Pakistan	Lahore	247.0	2014-01-01 13:00:00+00:00	8	female	0.0
4	400.0	400.0	Milk Sales	Food	Pakistan	Abdul Hakeem	245.0	2014-01-01 19:18:51+00:00	16	female	0.0

В рамках модуля plot библиотеки Pandas можно создать график плотности распределения, задав параметр kind='kde':

```
kiva_loans_pandas.unfunded_amount.plot(kind='kde')
```

```
<Axes: ylabel='Density'>
```

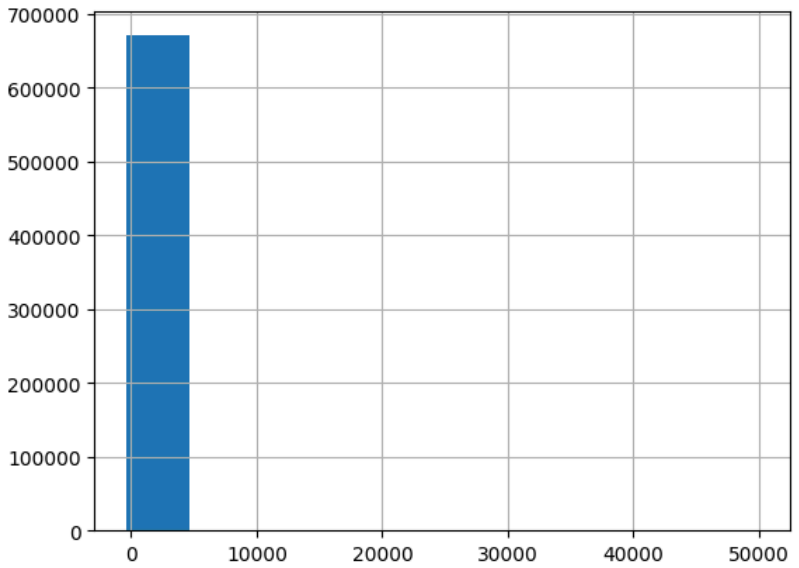


Судя по этому графику все значения равны нулю. Однако нам показан достаточно большой промежуток — от ~-25k до ~75k. Возможно, там находятся выбросы.

Построим гистограмму для той же переменной:

```
kiva_loans_pandas.unfunded_amount.hist()
```

```
<Axes: >
```

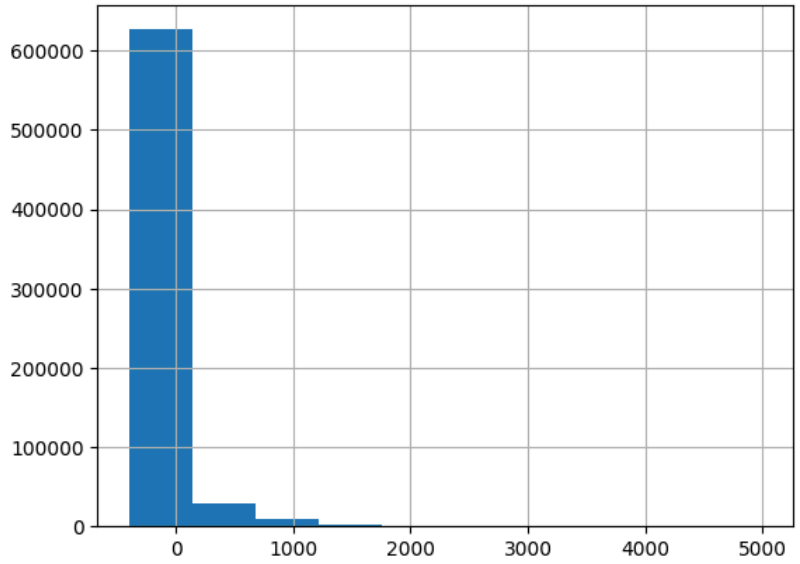


Та же ситуация — столбец один, а интервал показан огромный (до 50k).

Следовательно, надо избавиться от выбросов, которые портят визуализацию. Попробуем ограничиться пятью тысячами:

```
kiva_loans_pandas[kiva_loans_pandas['unfunded_amount'] <
5000]['unfunded_amount'].hist()
```

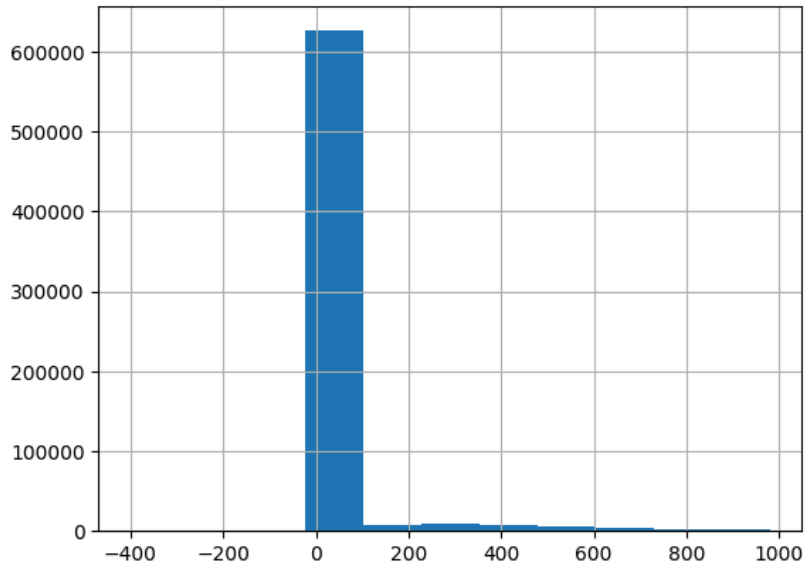
```
<Axes: >
```



Теперь мы видим больше столбцов. Попробуем ограничить визуализацию еще больше:

```
kiva_loans_pandas[kiva_loans_pandas['unfunded_amount'] <
1000]['unfunded_amount'].hist(bins=11)
```

```
<Axes: >
```



Стало лучше. Но теперь самый высокий столбик немного заходит в отрицательную плоскость. Нужно разобраться, есть ли какие-то значения или это особенность разметки гистограммы. Проверим на данных:

```
kiva_loans_pandas[kiva_loans_pandas['unfunded_amount'] <=
0]['unfunded_amount'].value_counts()
```

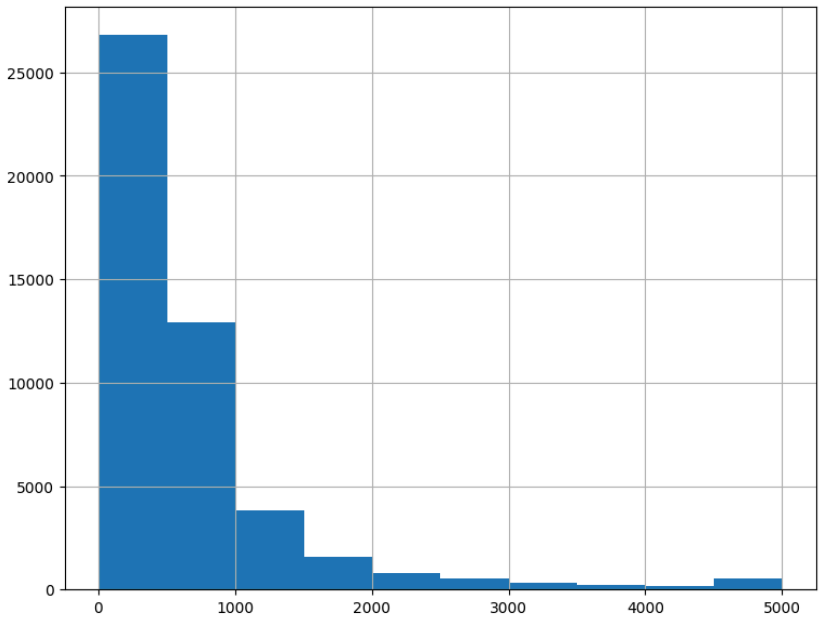
```
0.0       622875
-25.0          1
-400.0         1
Name: unfunded_amount, dtype: int64
```

Мы видим, что всего два значения заходят в отрицательную плоскость — это точно выбросы. При этом почти 623 тыс. значений равны нулю, что сильно портит визуализацию остальных результатов.

Построим гистограмму распределения значений в интервале (0, 5000]:

```
kiva_loans_pandas[(kiva_loans_pandas['unfunded_amount'] > 0) &
(kiva_loans_pandas['unfunded_amount'] <=
5000)]['unfunded_amount'].hist(figsize=(9, 7))
```

```
<Axes: >
```



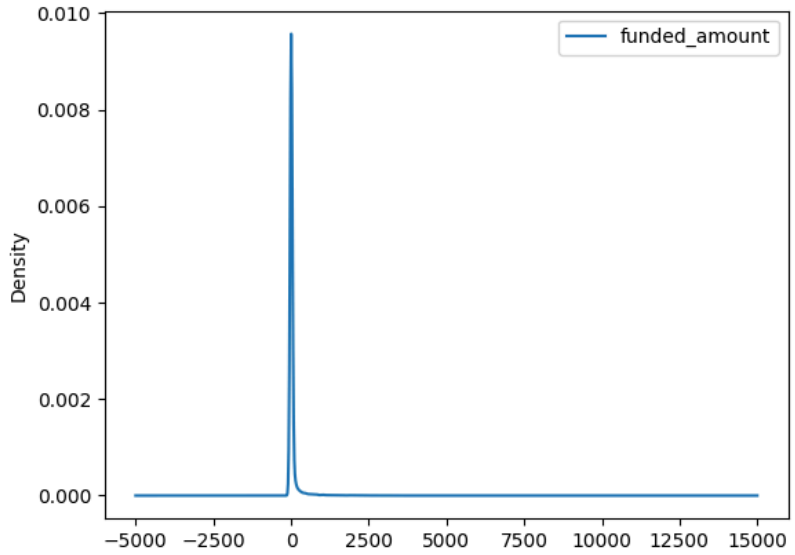
Таким образом, благодаря построению серии быстрых графиков мы смогли обнаружить интервалы, в которых находятся выбросы, и понять, как лучше визуализировать имеющиеся значения.

То же самое можно проделать при решении других задач. Например, посмотрим распределение количества субсидируемых регионов в каждой стране:

```
funded_amount_per_region =
kiva_loans_pandas.groupby(['country', 'region'],
as_index=False)['funded_amount'].count()
```

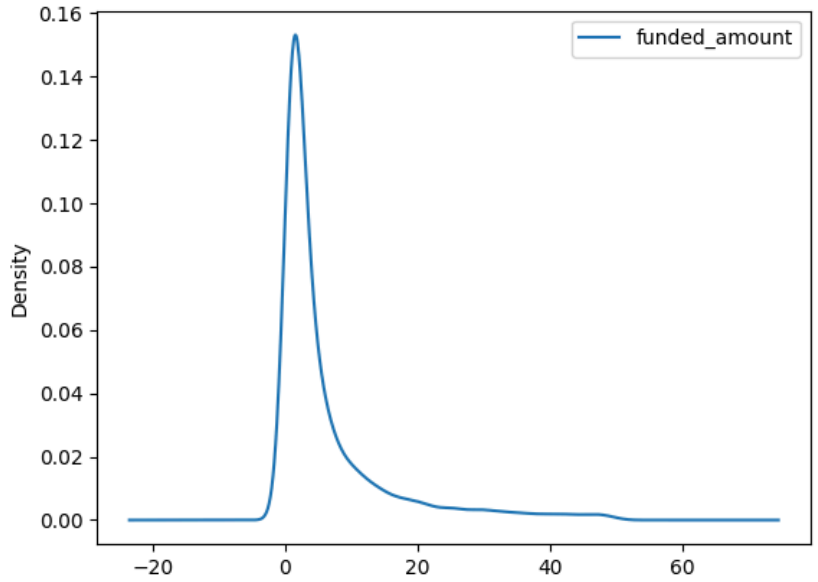
```
funded_amount_per_region.plot(kind='kde')
```

```
<Axes: ylabel='Density'>
```




```
funded_amount_per_region[funded_amount_per_region['funded_amou
nt'] <= 50].plot(kind='kde')
```

```
<Axes: ylabel='Density'>
```




```
funded_amount_per_region.groupby('country')['region'].count().
describe()
```

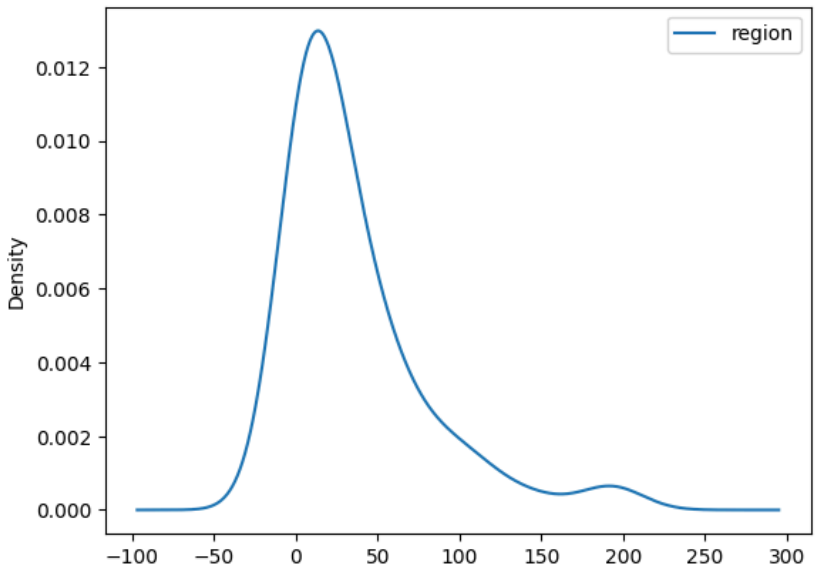
```
count       82.000000
```

```
mean       155.475610
std        429.059731
min          1.000000
25%          8.000000
50%         32.000000
75%        116.000000
max       3638.000000
Name: region, dtype: float64
```

```
regions_in_country =
funded_amount_per_region.groupby('country',
as_index=False)['region'].count()
```

```
regions_in_country[regions_in_country['region']<200].plot(kind
='kde')
```

```
<Axes: ylabel='Density'>
```



Метод .pipe(): использование собственного функционала Pandas

В Pandas существует метод .pipe(). Он позволяет использовать собственные функции для работы с Pandas. Таким образом, мы можем использовать все, что было написано ранее, более эффективным путем.

Например, мы написали функцию gender_get, которая принимает на вход столбец с гендерным составом группы, и применяет к каждому наблюдению алгоритм, возвращающий значения male, female или group. То же самое мы можем использовать с помощью Pandas-метода .pipe(), но в более удобном и практичном виде. Попробуем его применить:

```
# Снова скопируем наш датафрейм в отдельную переменную

kiva_loans_pipe =
pd.read_csv('https://raw.githubusercontent.com/ElijahSum/mipt_visualization/master/week_02_visualization/kiva_loans_week2',
index_col=0)
```

```
kiva_loans_pipe.head(5)
```

	funded_amount	loan_amount	activity	sector	country	region	partner_id	funded_time	lender_count	borrower_genders	
0	300.0	300.0	Fruits & Vegetables	Food	Pakistan	Lahore	247.0	2014-01-02 10:06:32+00:00	12	female	
1	575.0	575.0	Rickshaw	Transportation	Pakistan	Lahore	247.0	2014-01-02 09:17:23+00:00	14	female, female	

2	150.0	150.0	Transportation	Transportation	India	Maynaguri	334.0	2014-01-01 16:01:36+00:00	6	female
3	200.0	200.0	Embroidery	Arts	Pakistan	Lahore	247.0	2014-01-01 13:00:00+00:00	8	female
4	400.0	400.0	Milk Sales	Food	Pakistan	Abdul Hakeem	245.0	2014-01-01 19:18:51+00:00	16	female

Применим метод .pipe() с функцией gender_get к нашему «чистому» датафрейму:

```
kiva_loans_pipe['borrower_genders'] =
kiva_loans_pipe['borrower_genders'].pipe(gender_get)
```

```
kiva_loans_pipe.head(5)
```

funded_amount	loan_amount	activity	sector	country	region	partner_id	funded_time	lender_count	borrower_genders	
0	300.0	300.0	Fruits & Vegetables	Food	Pakistan	Lahore	247.0	2014-01-02 10:06:32+00:00	12	female
1	575.0	575.0	Rickshaw	Transportation	Pakistan	Lahore	247.0	2014-01-02 09:17:23+00:00	14	group

2	150.0	150.0	Transportation	Transportation	India	Maynaguri	334.0	2014-01-01 16:01:36+00:00	6	female
3	200.0	200.0	Embroidery	Arts	Pakistan	Lahore	247.0	2014-01-01 13:00:00+00:00	8	female
4	400.0	400.0	Milk Sales	Food	Pakistan	Abdul Hakeem	245.0	2014-01-01 19:18:51+00:00	16	female

Как мы видим, результат не отличается от ранее полученных. В чем же преимущество использования этого метода?

Метод .pipe() позволяет в более понятной и краткой манере применять несколько функций к датафрейму:

```
# Создадим функции, которые переведут количество денег в тысячи и добавят в конце значок доллара


def number_to_thousands(series):
 series /= 1000
 return series


def add_dollar_sign(series):
 series = series.apply(lambda x: str(x) + '$')
 return series


kiva_loans_pipe['loan_amount'].pipe(number_to_thousands).pipe(add_dollar_sign)
```

```
0            0.3$
1          0.575$
2           0.15$
3            0.2$
4            0.4$
            ...
671200    0.025$
671201    0.025$
671202    0.025$
671203    0.025$
671204    0.025$
Name: loan_amount, Length: 671205, dtype: object
```

Сравним читаемость:

```
kiva_loans_pipe['loan_amount'].pipe(number_to_thousands).pipe(
add_dollar_sign)
```

```
0            0.0003$
1          0.000575$
2           0.00015$
3            0.0002$
4            0.0004$
             ...
671200     2.5e-05$
671201     2.5e-05$
671202     2.5e-05$
671203     2.5e-05$
```

```
671204     2.5e-05$
Name: loan_amount, Length: 671205, dtype: object
```

```
add_dollar_sign(number_to_thousands(kiva_loans_pipe['loan_amou
nt']))
```

```
0                          3e-07$
1                       5.75e-07$
2                        1.5e-07$
3         2.0000000000000002e-07$
4         4.0000000000000003e-07$
                   ...
671200    2.5000000000000002e-08$
671201    2.5000000000000002e-08$
671202    2.5000000000000002e-08$
671203    2.5000000000000002e-08$
671204    2.5000000000000002e-08$
Name: loan_amount, Length: 671205, dtype: object
```

Отличия методов .apply() и .pipe

Метод .apply():

- **Применение к строкам или столбцам:** в основном используется для применения функции вдоль оси DataFrame, либо вдоль строк (axis=0), либо вдоль столбцов (axis=1).
- **Гибкость функций:** метод универсальный и может принимать функции, работающие с одномерными массивами, такими как объекты Series.

- **Операции вдоль столбцов или строк:** выполняет операции вдоль столбцов или строк, что делает его мощным инструментом для настраиваемых преобразований.
- **Доступ к нескольким столбцам:** функция, переданная .apply(), может обращаться к нескольким столбцам одновременно и возвращать Series или скалярное значение.

Метод .pipe():

- **Цепочка методов:** предназначен для цепочки методов, что позволяет более свободно объединять несколько операций.
- **Передача DataFrame в функцию:** принимает функцию в качестве аргумента, и эта функция может принимать и возвращать DataFrame. Это делает его удобным для более сложных манипуляций данными.
- **Ясность и читаемость кода:** повышает ясность и читаемость кода, когда есть последовательность операций с DataFrame.
- **Пользовательские функции:** можно определить пользовательские функции и применять их с использованием .pipe(), что полезно для создания многоразовых конвейеров обработки данных.

Самое полезное для нас сейчас — метод .pipe позволяет создавать и использовать шаблоны для наших графиков, которые можно использовать в анализе.

Выводы

Работа со временем:

- Для работы со временем есть специальные библиотеки. Самая популярная из них — time. Однако не очень удобно использовать эту библиотеку в отрыве от всего остального.
- В библиотеку Pandas уже встроены все необходимые методы и собственный тип данных для работы со временем TimeStamp.
- После импорта данных со столбцом дат, необходимо проверить тип данных этого столбца. Если даты импортировались некорректно, стоит использовать функцию pd.to_datetime().

Использование Pandas для визуализации:

- В библиотеке Pandas есть ряд методов для простейшей визуализации данных. Например, чтобы создать быструю визуализацию в ходе EDA, необязательно обращаться к другим библиотекам.
- Самые популярные типы визуализации можно создать напрямую, указав название графика (df.hist() или df.plot()). К другим можно обратиться лишь через подмодуль .plot (df.plot.bar() или df.plot.scatter()).
- С помощью встроенных методов Pandas можно строить визуализацию временных рядов, обозначив в качестве параметра x столбец с временем. Например, loans_per_month.plot.bar(x='YearMonth', y='funded_amount', figsize=(8, 5)).

Написание собственных функций для работы с Pandas:

- Написанную функцию можно применить к столбцам или строкам датафрейма с помощью метода .apply().
- Создание собственных функций может помочь при создании визуализаций в Pandas.
- Чтобы создать собственные функции для применения к нескольким столбцам или ко всему датасету, используется метод .pipe(). Он специально создан, чтобы максимально эффективно применять последовательно несколько функций к одному датасету.

Дополнительная литература

- Модуль Time
- struct_time: именованный кортеж